

LAPORAN AKHIR PROYEK DOCKER
MATA KULIAH ARSITEKTUR KOMPUTER DAN SISTEM
OPERASI



Anggota Kelompok:

Aginda Naufal Adil (25031554245)
Fasha Anandhikta Yanuara (25031554040)
Muhammad Alvian Alpha Romeo (25031554080)
Parama Zaidan Harandi (25031554157)

Dosen Pengampu :

Hasanuddin Al-Habib, M.Si
Sabrina Amelialevi, S.Kom., M.Kom.

UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN
ALAM / PRODI SAINS DATA
TAHUN 2025

DAFTAR ISI

Daftar Isi

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
BAB II ISI.....	4
2.1 Pembahasan dan Implementasi Konfigurasi Docker Compose.....	4
2.2 Pengujian API Menggunakan Postman.....	11
2.3 Penambahan Struktur pada main.py.....	16
2.3.1 Penjelasan File main.py.....	16
2.3.2 Perbedaan Kode Lama dan Kode Baru.....	18
2.4 Penambahan Struktur File Baru.....	19
BAB III KESIMPULAN.....	21
3.1 Kesimpulan.....	21
3.2 Rekomendasi.....	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan aplikasi modern menuntut arsitektur sistem yang modular, mudah dikembangkan, dan mudah diskalakan. Oleh karena itu, pendekatan arsitektur microservice digunakan dengan memisahkan layanan autentikasi dan layanan akademik agar setiap layanan dapat berjalan secara independen dan lebih mudah dipelihara. Layanan autentikasi dibangun menggunakan Node.js dan MongoDB untuk menangani pembuatan akun, proses login, serta penerbitan dan verifikasi JSON Web Token (JWT). Sementara itu, layanan akademik dibangun menggunakan FastAPI dan PostgreSQL untuk mengelola data mahasiswa serta menghitung Indeks Prestasi Semester (IPS) melalui endpoint API terpisah. Seluruh permintaan client diatur melalui API Gateway berbasis Nginx yang berfungsi sebagai single entry point dan reverse proxy. Seluruh layanan dijalankan dalam container menggunakan Docker dengan Docker Compose sebagai pengelola jaringan, dependensi, dan konfigurasi layanan, sehingga proses deployment dapat dilakukan secara konsisten dan terstruktur.

1.2 Tujuan

Tujuan dari pelaksanaan tugas ini adalah sebagai berikut :

- Menerapkan konsep arsitektur microservice pada layanan akademik sederhana.
- Mengonfigurasi dan menjalankan beberapa container menggunakan Docker Compose.
- Mengintegrasikan layanan autentikasi, basis data, dan API Gateway dalam satu sistem.
- Melatih kemampuan kerja tim dalam menyelesaikan proyek berbasis DevOps.

BAB II

PEMBAHASAN

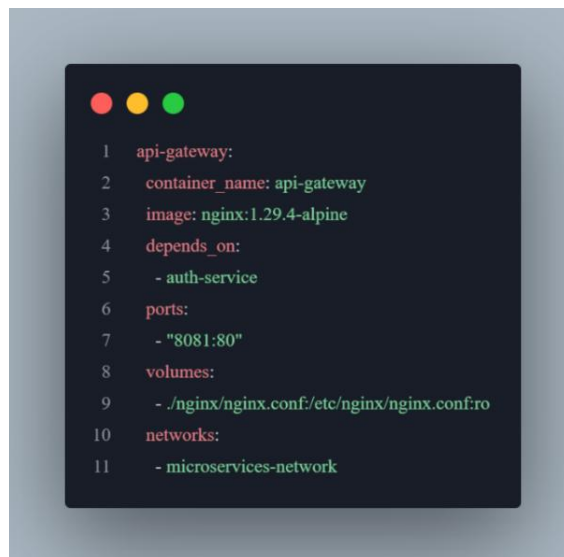
2.1 Pembahasan dan Implementasi Konfigurasi Docker Compose

Berdasarkan kerangka dasar (skeleton) yang diberikan, telah dilakukan modifikasi pada berkas docker compose. Sistem dibangun menggunakan lima microservice utama, sehingga setiap komponen berjalan sebagai container terpisah dan terhubung melalui jaringan internal:

- 1) api-gateway
- 2) auth-service
- 3) auth-db
- 4) acad-service
- 5) acad-db

1. API Gateway (Nginx)

API Gateway berperan sebagai reverse proxy yang menerima seluruh permintaan dari client dan meneruskannya ke layanan backend yang sesuai. Seluruh request dari client diarahkan melalui satu alamat, yaitu **localhost:8081**, sehingga client tidak berinteraksi langsung dengan masing-masing service.



Gambar 2.1

Penjelasan:

- Menggunakan image `nginx:alpine` karena ukurannya kecil dan efisien untuk dijadikan API Gateway.
- Port 8081 pada host dipetakan ke port 80 di dalam container agar layanan dapat diakses dari luar.
- Seluruh permintaan dari client atau Postman diarahkan melalui alamat **`http://localhost:8081/`**.
- File konfigurasi `nginx.conf` dipasang ke dalam container dengan mode `read-only` untuk mencegah perubahan konfigurasi secara langsung.
- Pengaturan routing memungkinkan pemisahan layanan, misalnya endpoint `/api/auth/` diteruskan ke `auth-service:3001` dan `/api/acad/` ke `acad-service:8000`.
- Opsi `depends_on` digunakan untuk memastikan API Gateway dijalankan setelah service backend terkait aktif.

2. Auth Service

Sistem ini terdiri dari dua backend yang terpisah, masing-masing memiliki peran dan tanggung jawab yang jelas.

a. Auth Service (Node.js)

- Bertanggung jawab atas proses autentikasi pengguna, meliputi login, registrasi, dan pengelolaan token **JWT**
- Terhubung dengan database **auth-db** menggunakan (**MongoDB**)
- Hanya menangani proses request yang berkaitan dengan otentikasi pengguna
- Menyediakan endpoint untuk melakukan verifikasi atau validasi token

b. Acad Service (FsstApi)

- Mengelola informasi akademik seperti data mahasiswa, mata kuliah, dan krs
- Melakukan perhitungan IPS berdasarkan data yang tersimpan di **PostgreSQL**
- Terhubung dengan database **acad-db** berbasis **PostgreSQL**

- Menyediakan endpoint untuk pengambilan dan pengelolaan data akademik



Gambar 2.2

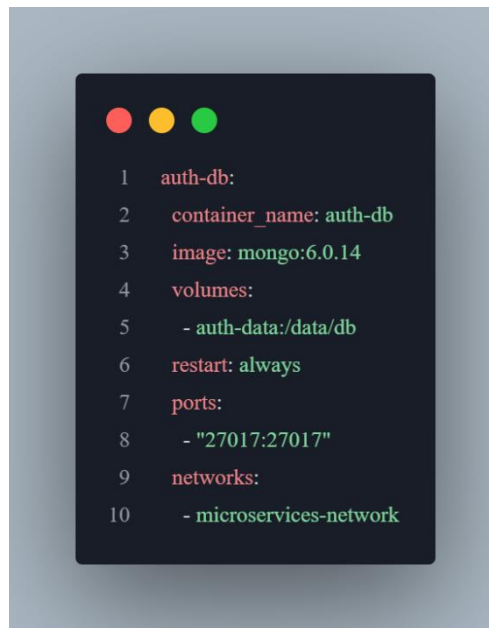
Penjelasan:

- Service ini dibangun menggunakan **Dockerfile** yang terdapat pada direktori **auth-service (Node.js)**.
- Penggunaan resource dibatasi hingga **0.5 core CPU** dan **512 MB RAM** guna menjaga efisiensi kinerja system.
- Konfigurasi **restart: always** diterapkan agar service dapat berjalan kembali secara otomatis apabila terjadi kegagalan.
- Koneksi ke database **MongoDB** dilakukan melalui URL **mongodb://auth-db:27017/auth**
- Variabel **JWT_SECRET** digunakan untuk proses penandatanganan serta verifikasi token.
- Volume dimanfaatkan pada tahap pengembangan sehingga perubahan pada kode dapat langsung diterapkan tanpa perlu melakukan build ulang.

3. Auth Database (MongoDB Auth)

Terdapat dua database yang dipisahkan untuk membedakan domain data:

- **auth-db:** Digunakan untuk menyimpan data pengguna yang dikelola oleh **Auth Service**.
- **acad-db:** Digunakan untuk menyimpan data akademik yang dikelola oleh **Acad Service**

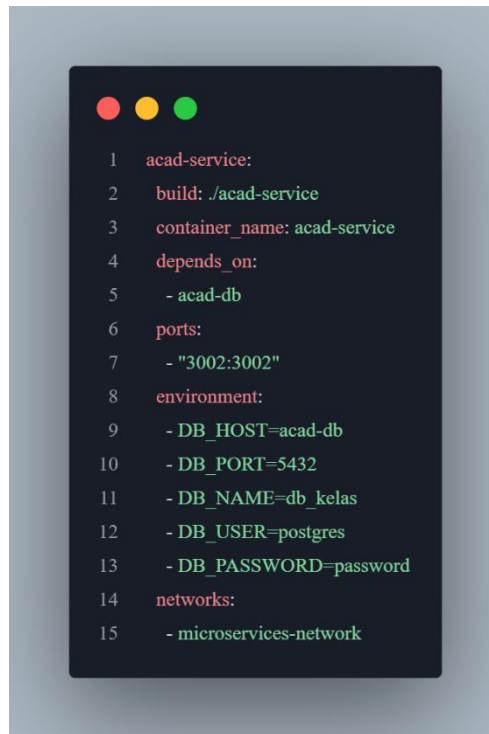


Gambar 2.3

Penjelasan:

- Menyimpan seluruh data pengguna yang dibutuhkan oleh **Auth Service**, seperti kredensial dan profil pengguna
- Menggunakan volume **auth-data** yang bersifat persisten, sehingga data tetap tersimpan meskipun container dihapus
- Terhubung dalam jaringan internal yang sama, yaitu **microservices-network**
- **Auth Service** dapat mengakses database menggunakan hostname **auth-db** melalui mekanisme DNS internal Docker
- Menggunakan image **MongoDB versi 6.0.14** yang dipilih karena stabilitas serta dukungan fitur keamanannya

4. Acad Service



Gambar 2.4

Penjelasan:

- Port 3002 diekspos agar acad-service dapat diakses langsung untuk keperluan debugging tanpa harus melalui API Gateway.
- Konfigurasi environment variables digunakan untuk mengarahkan koneksi ke kontainer basis data acad-db pada port 5432 beserta kredensial yang sesuai.
- Opsi depends_on memastikan acad-service hanya dijalankan setelah kontainer acad-db aktif sehingga koneksi database lebih stabil.
- Perhitungan IPS dilakukan dengan langkah-langkah berikut:
 - Melakukan query join antara tabel mahasiswa, krs, dan mata_kuliah.
 - Mengonversi nilai huruf menjadi bobot angka berdasarkan skala yang telah ditentukan.
 - Menghitung IPS menggunakan rumus total (bobot × SKS) dibagi total SKS untuk mendapatkan nilai IPS akhir.

5. Acad Database

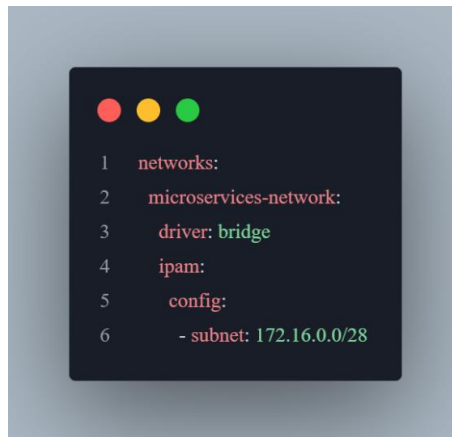


Gambar 2.5

Penjelasan:

- Komponen acad-db bertindak sebagai lapisan persistensi data untuk layanan akademik.
- Menggunakan image PostgreSQL 16.2 berbasis Alpine Linux sehingga lebih ringan dan efisien dalam penggunaan sumber daya.
- Basis data dikonfigurasi dengan volume bind-mount ke path `./docker-entrypoint-initdb.d` di dalam kontainer, sehingga skema tabel dapat diinjeksi secara otomatis saat deployment awal menggunakan skrip inisialisasi.
- Komunikasi antara acad-service dan acad-db dilakukan melalui jaringan internal *microservices-network*, sedangkan port 5432 diekspos ke host untuk keperluan debugging maupun manajemen data secara manual (misalnya akses via client PostgreSQL).

6. Konfigurasi Internal Networking

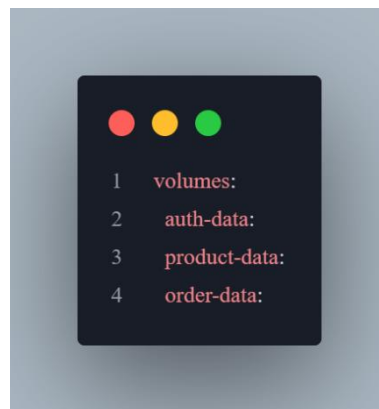


Gambar 2.6

Penjelasan:

- Semua service berada dalam 1 jaringan.
- Menggunakan driver bridge.
- Subnet ditentukan manual, sehingga setiap container mendapatkan IP internal yang konsisten.

7. Konfigurasi Volumes



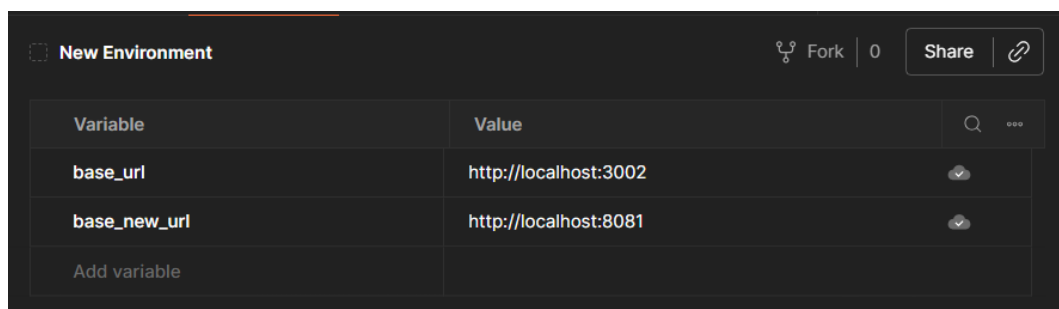
Gambar 2.7

Penjelasan:

- auth-data dan acad-data digunakan dalam proyek ini.
- product-data dan order-data tampaknya disiapkan untuk pengembangan ke depan, namun tidak digunakan dalam service aktif.

2.2 Pengujian API Menggunakan Postman

Setelah seluruh service berhasil dijalankan dengan Docker Compose, tahap berikutnya adalah melakukan pengujian API menggunakan Postman. Pengujian ini bertujuan untuk memastikan bahwa setiap endpoint yang telah dibuat dapat berjalan dengan baik dan memberikan respons sesuai dengan fungsinya. Seluruh request dikirim melalui alamat `http://localhost:8081/` karena API Gateway dikonfigurasi untuk berjalan pada port 8081 sebagai pintu masuk utama sistem.



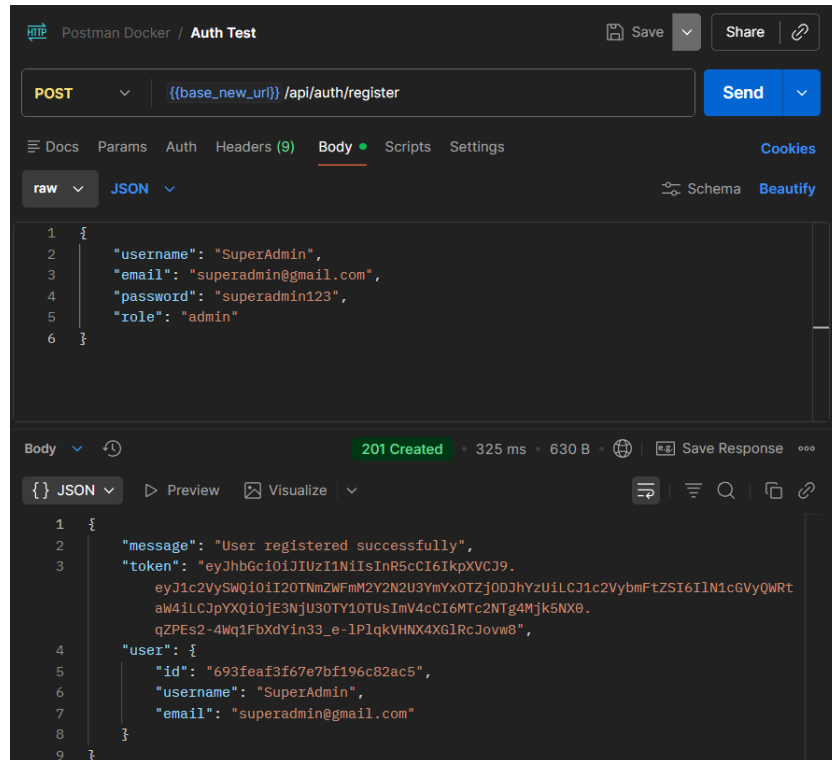
Gambar 2.8 (Environment)

Penjelasan:

- Pada Postman dibuat sebuah environment dengan dua variabel: *base_url* = `http://localhost:3002` untuk mengakses acad-service secara langsung, dan *base_new_url* = `http://localhost:8081` untuk mengakses seluruh layanan melalui API Gateway.
- Penggunaan variabel environment ini membuat endpoint di koleksi cukup ditulis sebagai `{{base_url}}/api/acad/mahasiswa` atau `{{base_new_url}}/api/auth/login`, sehingga ketika port gateway berubah (misalnya dari 8080 ke 8081) cukup mengubah nilai variabel tanpa mengedit satu per satu URL request.

1. Pengujian Endpoint Auth Service

- Endpoint Register

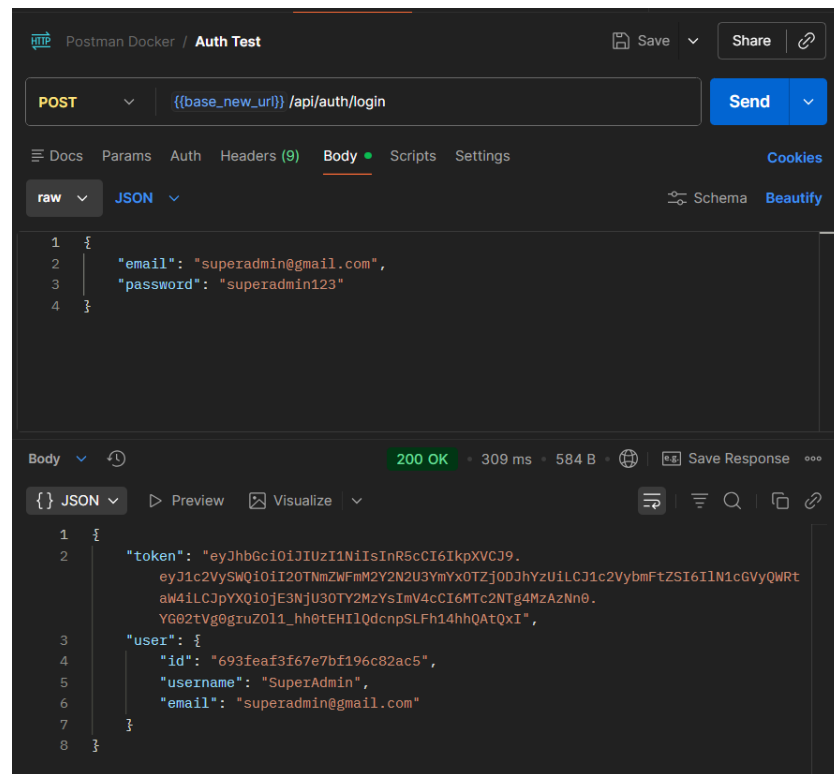


Gambar 2.9 (Register)

Penjelasan:

- Permintaan **POST** dikirim ke endpoint `/api/auth/register` melalui API Gateway menggunakan Postman.
- Body request berisi data `'username'`, `'email'`, `'password'`, dan `'role'` yang digunakan untuk membuat akun pengguna baru.
- Layanan auth-service memeriksa apakah email atau username sudah terdaftar, melakukan hashing password dengan bcrypt, kemudian menyimpan data pengguna ke MongoDB.
- Jika berhasil, layanan mengembalikan respons 201 Created yang berisi pesan keberhasilan, token JWT, dan informasi pengguna yang baru terdaftar.

- Endpoint Login



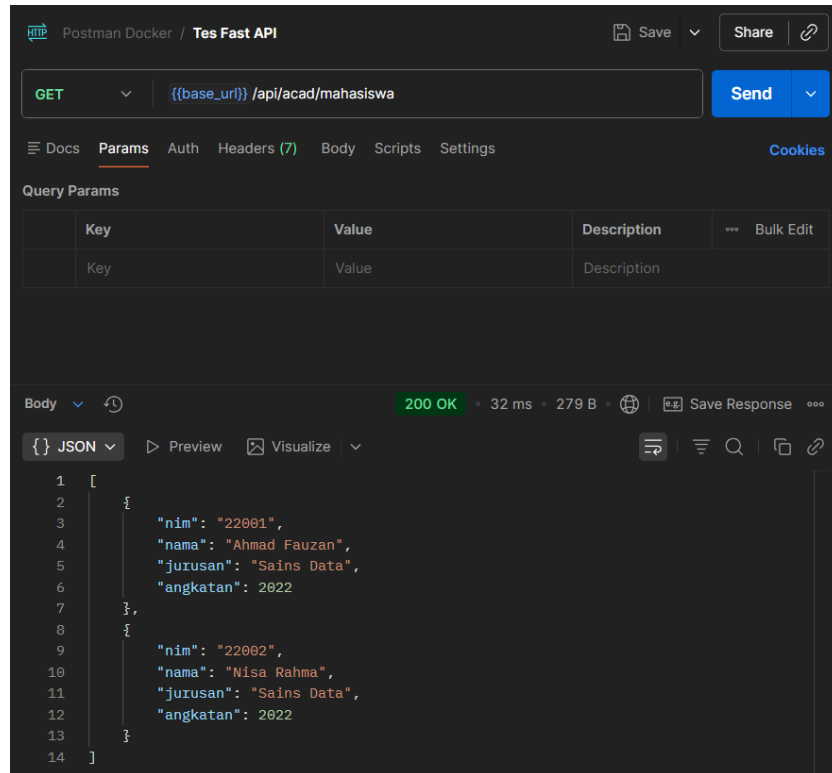
Gambar 2.10 (Login)

Penjelasan:

- Permintaan POST dikirim ke endpoint /api/auth/login dengan body JSON yang hanya berisi email dan password pengguna yang sudah terdaftar.
- Layanan auth-service mencari pengguna berdasarkan email, lalu membandingkan password yang dikirim dengan password yang sudah di-hash di database menggunakan bcrypt.
- Jika kredensial valid, layanan menghasilkan token JWT baru dan mengembalikannya bersama informasi pengguna dalam respons 200 OK.

2. Pengujian Endpoint Acad Service

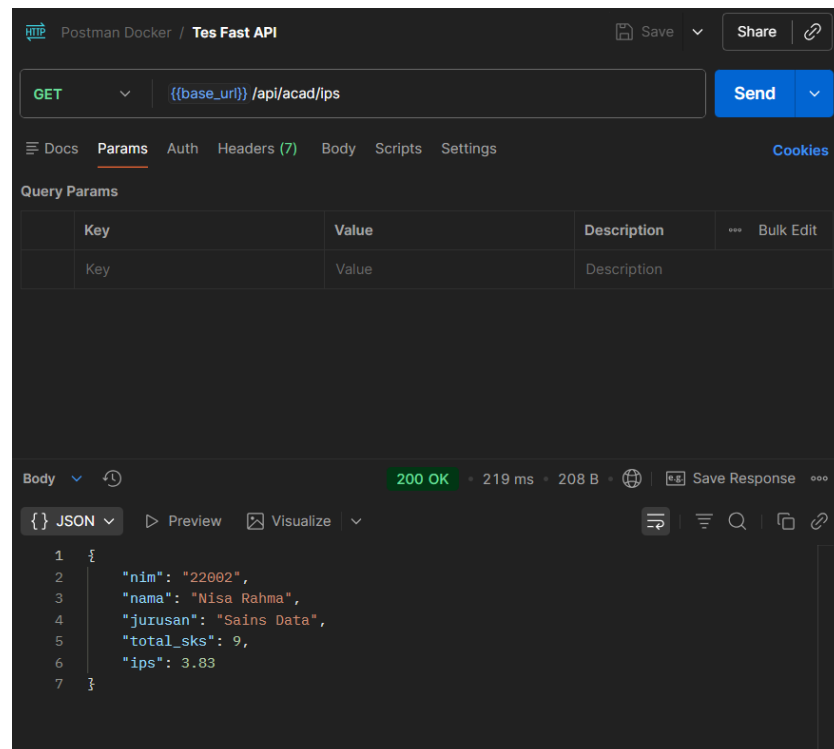
- **Endpoint GET `/api/acad/mahasiswa`**



Gambar 2.11 (GET `/api/acad/mahasiswa`)

Pengujian endpoint `GET /api/acad/mahasiswa` dilakukan melalui Postman dengan URL `{{base_url}}/api/acad/mahasiswa`, yang mengarah langsung ke acad-service pada port 3002. Endpoint ini mengembalikan daftar data mahasiswa dalam bentuk JSON (nim, nama, jurusan, angkatan), yang menunjukkan bahwa koneksi antara acad-service dan basis data acad-db telah berjalan dengan benar.

- **Endpoint GET `/api/acad/ips`**



Gambar 2.12 (GET `/api/acad/ips`)

Pengujian endpoint `GET /api/acad/ips` dilakukan dengan URL `{{base_url}}/api/acad/ips`, dan menghasilkan respons JSON berisi nim, nama, jurusan, total_sks, serta nilai ips yang telah dihitung. Nilai IPS tersebut merupakan hasil dari query join beberapa tabel dan perhitungan total ($\text{bobot} \times \text{SKS}$) dibagi total SKS, sehingga pengujian ini sekaligus memverifikasi bahwa logika bisnis perhitungan IPS di acad-service berfungsi sesuai rancangan.

2.3 Penambahan Struktur pada main.py

2.3.1 Penjelasan File main.py

1. Deskripsi Umum

File main.py berfungsi sebagai entry point aplikasi backend yang dibangun menggunakan FastAPI. File ini mengatur konfigurasi aplikasi, model data, koneksi ke database PostgreSQL, serta mendefinisikan endpoint yang digunakan untuk pengambilan data mahasiswa dan perhitungan Indeks Prestasi Semester (IPS).

2. Import Library

Kode dimulai dengan mengimpor beberapa modul penting, yaitu:

- a. FastAPI, HTTPException, dan Query untuk pembuatan API, validasi input, dan penanganan error.
- b. CORSMiddleware untuk mengaktifkan akses lintas domain agar API dapat dipanggil oleh layanan lain.
- c. BaseModel dan Field dari Pydantic untuk validasi data.
- d. Optional dan List dari typing untuk kebutuhan tipe data tambahan.
- e. Datetime untuk mencatat waktu proses dalam beberapa endpoint.
- f. Psycopg2 dan RealDictCursor untuk koneksi dan eksekusi query PostgreSQL.
- g. Contextmanager untuk mengelola transaksi database secara aman.

3. Struktur Model Data

Model Pydantic digunakan untuk memastikan data request terstruktur dengan baik. Contohnya model Mahasiswa yang berisi atribut nim, nama, jurusan, dan angkatan. Validasi dilakukan otomatis ketika data masuk, sehingga meminimalkan kesalahan.

4. Inisialisasi Aplikasi (BETOLLL)

Aplikasi diinisialisasi dengan FastAPI(). Kemudian, middleware CORS ditambahkan untuk mengizinkan permintaan dari seluruh

domain, terutama ketika API dijalankan bersama frontend atau service lain melalui Docker.

5. Koneksi dan Transaksi Database

Aplikasi mengonfigurasi akses ke database PostgreSQL melalui penggunaan environment variable (`os.getenv`). Context manager `get_db_connection()` digunakan untuk memastikan koneksi terbuka, transaksi dilakukan, dan koneksi selalu ditutup dengan benar, bahkan jika terjadi error.

6. Endpoint Perhitungan IPS

Endpoint `/api/acad/ips` bertanggung jawab untuk menghitung IPS mahasiswa berdasarkan data nyata dari database. Proses perhitungan IPS meliputi:

- a. Mengambil data nilai mata kuliah dan jumlah SKS dari tabel relasional (mahasiswa, krs, dan mata_kuliah).
- b. Mengubah nilai huruf menjadi bobot angka ($A = 4$, $B+ = 3.5$, $B = 3$, dan seterusnya).
- c. Mengalikan bobot angka dengan jumlah SKS setiap mata kuliah.
- d. Menjumlahkan total bobot seluruh mata kuliah.
- e. Menghitung IPS dengan membagi total bobot dengan total SKS.
- f. Mengembalikan hasil IPS dalam format JSON.

7. Penanganan Error

Jika terjadi kesalahan pada koneksi atau query database, sistem akan melempar `HTTPException` dengan kode status 500. Ini memastikan informasi error disampaikan dengan jelas kepada layanan atau pengguna yang memanggil API.

8. Garis besar dalam file main.py

Secara garis besar, file `main.py` memiliki peran penting dalam:

- a. Struktur arsitektur API
- b. Validasi input
- c. Penyediaan data mahasiswa
- d. Perhitungan IPS berbasis database

- e. Penanganan error
- f. Integrasi dengan Docker melalui environment variable
- g. Keamanan dan konsistensi transaksi database

2.3.2 Perbedaan Kode Lama dan Kode Baru

1. Penggunaan Database

Kode lama:

- a. Tidak terhubung ke database.
- b. Data nilai dikirim manual melalui request.

Kode baru :

- a. Terhubung langsung ke PostgreSQL.
- b. Data diambil melalui query SQL dari tabel mahasiswa, krs, dan mata kuliah.
- c. Menggunakan context manager untuk transaksi yang aman.

2. Mekanisme Perhitungan IPS

Kode lama:

- a. Input nilai dan SKS dikirim dari frontend.
- b. Semua perhitungan dilakukan sepenuhnya berdasarkan input manual.

Kode Baru:

- a. Perhitungan IPS didasarkan pada data nyata dari database.
- b. Setiap nilai huruf dan SKS diambil langsung dari tabel terkait sehingga hasil lebih akurat dan konsisten.

3. Struktur Endpoint:

Kode lama:

- a. Biasanya hanya ada satu endpoint utama untuk hitung IPS.

Kode baru:

- a. Memiliki beberapa endpoint seperti /health, /api/acad/mahasiswa, dan /api/acad/ips.
- b. Arsitektur lebih modular dan mengikuti standar REST.

4. Validasi Model Data

Kode lama:

- a. Pydantic digunakan untuk memvalidasi input manual

Kode baru:

- a. Pydantic digunakan untuk struktur data API, sementara validasi nilai diambil dari database dan dicek kembali dalam proses perhitungan.

5. Middlewere dan Konfigurasi

Kode lama:

- a. Konfigurasi CORS sering minimal atau belum dibuat.

Kode baru:

- a. CORS diaktifkan penuh untuk mendukung integrasi lintas service.
- b. Menggunakan environment variable agar mudah di-deploy melalui Docker.

6. Integrasi Docker

Kode lama:

- a.. Umumnya hanya berjalan lokal tanpa dukungan container.

Kode Baru:

- a. Parameter database dibaca dari environment Docker.
- b. Lebih siap digunakan dalam sistem terdistribusi dan microservices.

2.4 Penambahan Struktur File Baru

Pada berkas awal dari dosen, struktur proyek hanya berisi:

- Folder acad-service
- Folder auth-service
- Satu folder nginx yang berisi nginx.conf
- Satu berkas docker-compose.yml di root

Setelah dikerjakan dan disesuaikan dengan kebutuhan deployment microservice berbasis Docker, dilakukan beberapa penambahan dan penataan ulang sebagai berikut.

1. Folder api-gateway/ dan penempatan ulang nginx.conf
File nginx.conf yang semula berada di dalam folder nginx dipindahkan ke folder baru api-gateway/nginx.conf. Dengan cara ini, API Gateway diperlakukan sebagai satu service tersendiri yang punya direktori konfigurasi sendiri. Hal ini memudahkan saat file tersebut di-mount ke container api gateway melalui docker-compose.yml.
2. Folder auth-data/ sebagai volume untuk basis data Auth
Folder auth-data/ ditambahkan di level root sebagai direktori host yang dipetakan ke /data/db di dalam container auth-db. Mapping ini digunakan sebagai volume agar data MongoDB untuk Auth Service tetap tersimpan (persisten) meskipun container dihentikan atau dijalankan ulang.
3. Folder mongo-init/ dan berkas init-mongo.js
Folder mongo-init/ ditambahkan di root dan di dalamnya dibuat file ini mongo.js. File ini berisi skrip untuk menginisialisasi database Acad di MongoDB. Isi init-mongo.js dibuat dengan mengadaptasi struktur dan data dari db_kelas.sql, sehingga koleksi mahasiswa, mata_kuliah, dan krs langsung terisi saat pertama kali acad-db dijalankan. Pada docker compose.yml, folder mongo-init/ di-mount ke /docker-entrypoint-initdb.d pada service acad-db, sehingga skrip init-mongo.js otomatis dijalankan oleh image resmi Mongo ketika container dibuat pertama kali.
4. Penataan ulang root proyek menjadi direktori Microservice_ips_mahasiswa
Seluruh komponen (acad-service, auth-service, api-gateway, auth-data, mongo-init, dan docker-compose.yml) dikumpulkan dalam satu folder utama Microservice_ips_mahasiswa. Penataan ini memudahkan proses pengelolaan dan menjalankan perintah docker-compose up -d dari satu direktori kerja yang konsisten. Dengan demikian, penambahan file dan folder tidak hanya mengubah tampilan struktur proyek, tetapi langsung terkait dengan kebutuhan Docker: penggunaan volume, inisialisasi database otomatis melalui /docker-entrypoint-initdb.d, dan pemisahan API Gateway sebagai service terpisah dalam arsitektur microservice

BAB III

KESIMPULAN

3.1 Kesimpulan

Implementasi arsitektur microservice pada sistem akademik berhasil dilakukan dengan memisahkan layanan autentikasi dan layanan akademik ke dalam service yang berdiri sendiri dan memiliki database masing-masing. Penggunaan Docker Compose mempermudah proses deployment, pengelolaan container, serta komunikasi antar service melalui jaringan internal.

API Gateway berbasis Nginx berfungsi dengan baik sebagai pintu masuk utama yang mengatur routing request dari client ke service terkait. Selain itu, layanan akademik berhasil mengimplementasikan perhitungan Indeks Prestasi Semester (IPS) secara akurat menggunakan FastAPI dan database relasional.

3.2 Rekomendasi

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan penambahan validasi keamanan berbasis JWT, penerapan HTTPS dan rate limiting pada API Gateway, serta implementasi logging dan monitoring. Selain itu, optimalisasi performa dan pengembangan fitur akademik lanjutan dapat dilakukan agar sistem lebih siap digunakan pada skala yang lebih besar.

Folder project terlampir pada link google drive berikut:

https://drive.google.com/drive/folders/1gZEHQ2IUrSQgkIW6MQAfKEMs_aRxJTMa?usp=drive_link